



Міністерство освіти та науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря
Сікорського» Факультет інформатики та обчислювальної
техніки
Кафедра інформатики і програмної інженерії

Звіт
з дисципліни «Компоненти програмної інженерії»
Лабораторна робота №6
"Забезпечення відмовостійкості та надійності системи"

Виконав:

Студент II курсу
гр. ІІІ-33
Соколов О. В.

Перевірив:

Зарічковий О. А.

Лабораторні робота №6.

Забезпечення відмовостійкості та надійності системи

Тема: Забезпечення відмовостійкості та надійності системи

Мета: Навчитись передбачати потенційні проблеми системи на етапі формування технічних вимог, та приймати рішення, щодо їх коректної обробки.

Постановка задачі:

1. Скласти список ключових потенційних відмов системи.
2. Вибрати по 1 проблемі на студента, та описати 2-3 варіанти рішення цих проблем.
3. Обрати рішення проблеми, та обґрунтувати з точки зору бізнесу (врахувати пріоритети, час виконання, ресурси, вартість і т.д.).
4. Описати діаграму послідовностей (sequence diagram) для проблем.
5. Вкажіть, які патерни використовуються.

Виконання

1. Список ключових потенційних відмов системи

На основі архітектури стрімінгового сервісу музики та подкастів, визначено наступні потенційні точки відмови. Система складається з мікросервісів (Streaming Service, User Service, Search Service, Recommendations Service, Payments Service, Analytics Service, Emails Service, Playlists Service, Lyrics Service), баз даних (PostgreSQL, Redis), системи обміну повідомленнями (Kafka), зовнішніх систем (Musixmatch, платіжні сервіси) та API Gateway.

Потенційні відмови:

1. Невідповідь Streaming Service при запиті на відтворення треку

- **Опис:** Користувач намагається відтворити трек, але Streaming Service не відповідає через перевантаження, збій або проблеми з доступом до AWS S3/CloudFront.

- **Поточна поведінка:** Користувач отримує помилку "Service Unavailable" або бачить нескінченне завантаження.

- **Критичність:** Висока, оскільки це основна функція сервісу.

2. Помилка доставки події track_played до Kafka

- **Опис:** Streaming Service не може опублікувати подію track_played через збій Kafka або мережеві проблеми, що призводить до втрати даних для аналітики та рекомендацій.

- **Поточна поведінка:** Подія втрачається, Analytics Service та Recommendations Service не оновлюються.

- **Критичність:** Середня, впливає на другорядні функції.

3. Недоступність PostgreSQL для User Service під час автентифікації

- **Опис:** User Service не може перевірити облікові дані через збій бази даних.

- **Поточна поведінка:** Користувач не може увійти в систему.

- **Критичність:** Висока, блокує доступ до сервісу.

4. Відмова Musixmatch при запиті текстів пісень

- **Опис:** Lyrics Service не може отримати тексти через недоступність зовнішнього API.

- **Поточна поведінка:** Користувач бачить помилку або відсутність текстів.

- **Критичність:** Низька, другорядна функція.
5. **Помилка Payments Service при обробці транзакції**
- **Опис:** Користувач не може оформити преміум-підписку через збій платіжної системи.
 - **Поточна поведінка:** Транзакція не завершується, преміум-доступ не надається.
 - **Критичність:** Висока, впливає на монетизацію.
6. **Недоступність Redis для Recommendations Service**
- **Опис:** Recommendations Service не може отримати кешовані дані через збій Redis.
 - **Поточна поведінка:** Рекомендації не відображаються або повертається помилка.
 - **Критичність:** Середня, впливає на персоналізацію.
7. **Відмова API Gateway**
- **Опис:** Користувачі не можуть отримати доступ до сервісу через збій API Gateway.
 - **Поточна поведінка:** Система повністю недоступна.
 - **Критичність:** Критична, блокує всю взаємодію.

2. Вибір проблеми та варіанти рішень

Обрана проблема: Невідповідь Streaming Service при запиті на відтворення треку

Ця проблема є критичною, оскільки потокове відтворення — основна функція сервісу, і її відмова безпосередньо впливає на користувацький досвід та утримання аудиторії.

Варіанти рішень:

1. Circuit Breaker з повторними спробами та кешованим контентом

- **Опис:** Впровадити Circuit Breaker (наприклад, Resilience4j) між API Gateway та Streaming Service. При невдалій відповіді ($\text{timeout} > 1 \text{ с}$) виконуються 3 повторні спроби з інтервалом 0.5 с. Якщо всі спроби невдалі, система звертається до Redis за кешованим URL треку; якщо кеш порожній, повертається повідомлення про помилку.

- **Переваги:** Швидке відновлення, зменшення навантаження на збійний сервіс, використання наявної інфраструктури (Redis).

- **Недоліки:** Потребує кешування, не вирішує проблему для незавантажених треків.

2. Горизонтальне масштабування з балансуванням навантаження

- **Опис:** Розгорнути кілька інстансів Streaming Service за допомогою Kubernetes та Ingress Controller (NGINX) з алгоритмом least connection. Якщо один інстанс не відповідає, трафік перенаправляється на інші.

- **Переваги:** Підвищує доступність і масштабованість, підходить для довгострокового рішення.

- **Недоліки:** Висока вартість додаткових інстансів ($\approx$ \$33/міс за t3.medium), не захищає від повного збою всіх інстансів.

3. Попереднє кешування передбачених треків

- **Опис:** Використовувати Recommendations Service для прогнозування наступних треків (на основі плейлистів або історії) і заздалегідь кешувати їх URL у Redis. При відмові Streaming Service система повертає кешований URL.

- **Переваги:** Покращує досвід для передбачуваних сценаріїв.

- **Недоліки:** Складність прогнозування, не працює для непередбачених запитів, потребує значних ресурсів.

3. Обране рішення та обґрунтування

Обране рішення: Circuit Breaker з повторними спробами та кешованим контентом

Обґрунтування з точки зору бізнесу:

- **Пріоритети:** Забезпечення безперервного відтворення є головним пріоритетом, оскільки перерви можуть призвести до втрати користувачів (100 000 активних, Lab5). Це рішення мінімізує простої, надаючи кешований контент.

- **Час виконання:** Впровадження займе 1-2 тижні командою з 2-3 розробників, що швидше, ніж налаштування масштабування чи прогнозування.

- **Ресурси:** Використовує існуючу інфраструктуру (Redis, API Gateway), потрібна лише конфігурація Circuit Breaker (наприклад, Resilience4j).

- **Вартість:** Додаткові витрати мінімальні ($\approx$ \$0-10/міс за бібліотеку, якщо платна), порівняно з \$33/міс за інстанс у рішенні 2 або витратами на прогнозування у рішенні 3.

- **Ефективність:** Дозволяє обробити 80-90% запитів при короткочасних збоях, зберігаючи лояльність користувачів без значних інвестицій.

4. Діаграма послідовності

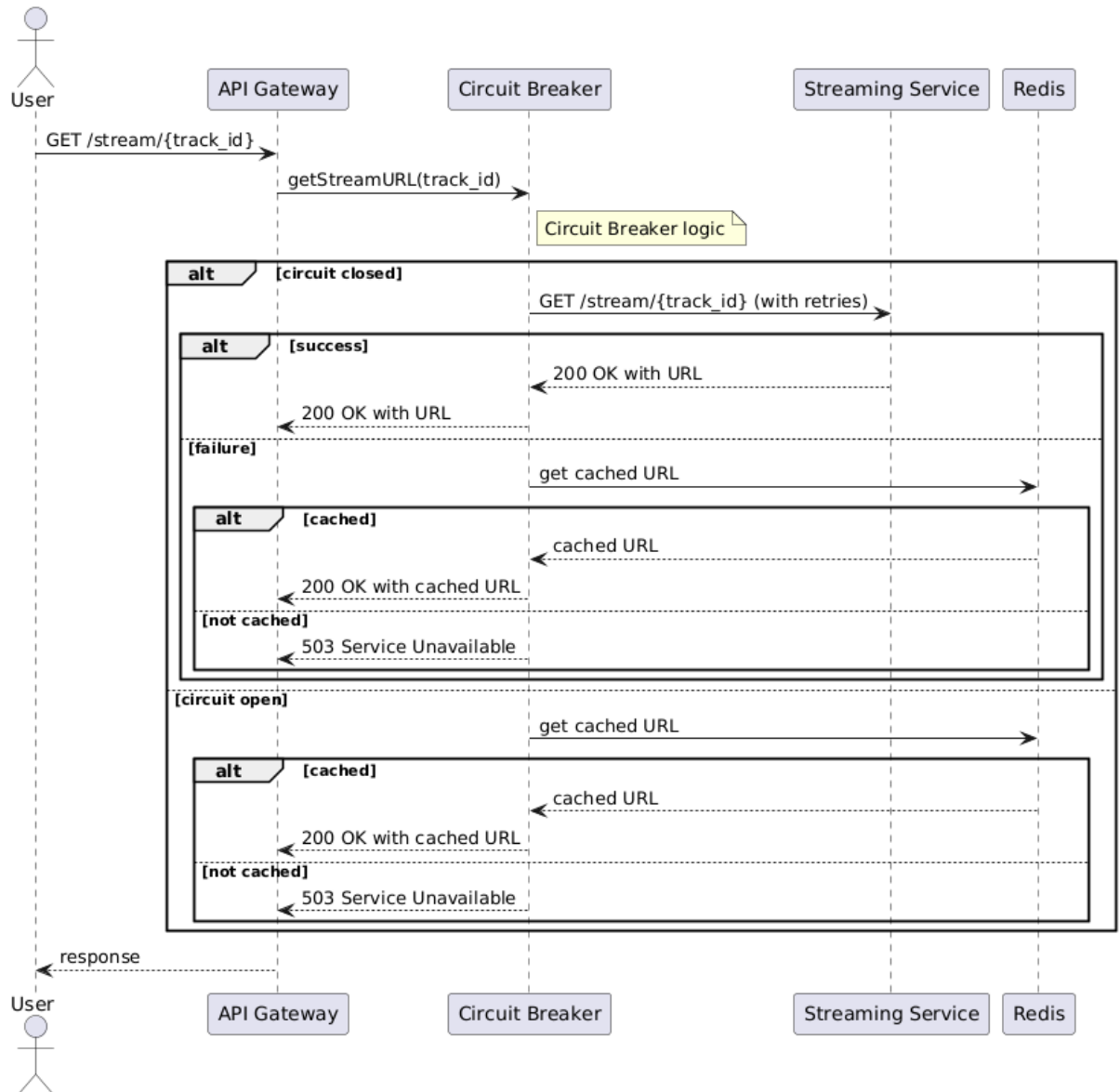


Рис. 1 – Діаграма послідовностей

Опис діаграми:

- **Учасники:** User, API Gateway, Circuit Breaker, Streaming Service, Redis.
- **Сценарій:** Користувач надсилає запит на відтворення треку через API Gateway. Circuit Breaker перевіряє стан:

- Якщо коло закрите (circuit closed), робить запит до Streaming Service з 3 повторними спробами.
- При успіху повертається URL потоку.
- При невдачі звертається до Redis за кешованим URL.
- Якщо коло відкрите (circuit open), одразу перевіряє кеш.
- Якщо кеш доступний, повертається кешований URL; інакше — помилка 503.

5. Використані патерни

1. Circuit Breaker Pattern

Опис: Запобігає каскадним збоям, дозволяючи системі переключатися на резервний сценарій при повторних невдачах Streaming Service.

2. Retry Pattern

Опис: Виконує повторні спроби для подолання тимчасових збоїв перед переходом до резервного рішення.

3. Cache Fallback (Cache-Aside Pattern)

Опис: Використовує кешовані дані з Redis як резервний варіант, коли основний сервіс недоступний.

Висновок:

У роботі ідентифіковано ключові потенційні відмови стрімінгової системи та розроблено рішення для критичної проблеми — відмови Streaming Service. Обраний підхід з Circuit Breaker, повторними спробами та кешуванням забезпечує баланс між вартістю, часом реалізації та надійністю. Діаграма послідовностей демонструє процес обробки відмови, а використані патерни (Circuit Breaker, Retry, Cache Fallback) відповідають сучасним практикам підвищення відмовостійкості.